Finding Bugs and Features Using Cryptographically-Informed Functional Testing

Giacomo Fenzi[1], Jan Gilcher[2] and Fernando Virdia[3]

[1] Computational Security Lab, EPFL, Lausanne, Switzerland
giacomo.fenzi@epfl.ch
[2] Applied Cryptography Group, ETH Zurich, Zurich, Switzerland
jan.gilcher@inf.ethz.ch
[3] University of Surrey, Guildford, United Kingdom
f.virdia@surrey.ac.uk

Abstract. In 2018, Mouha *et al.* (IEEE Trans. Reliability, 2018) performed a post-mortem investigation of the correctness of reference implementations submitted to the SHA3 competition run by NIST, finding previously unidentified bugs in a significant portion of them, including two of the five finalists. Their innovative approach allowed them to identify the presence of such bugs in a black-box manner, by searching for counterexamples of expected cryptographic properties of the implementations under test. In this work, we extend their approach to key encapsulation mechanisms (KEMs) and digital signature schemes (DSSs). We perform our tests on multiple versions of the LibOQS collection of post-quantum schemes to capture implementations at different points of the recent Post-Quantum Cryptography Standardization Process run by NIST. We identify multiple bugs, ranging from software bugs (segmentation faults, memory overflows) to cryptographic bugs, such as ciphertext malleability in KEMs claiming IND-CCA security. We also observe various features of KEMs and DSSs that do not contradict any security guarantees but could appear counter-intuitive. Finally, we compare this methodology with a traditional fuzzing campaign against LibOQS and SUPERCOP, observing that traditional fuzzing harnesses appear less effective in surfacing software and logical bugs.

Keywords: cryptographic implementation testing · metamorphic testing · implementation correctness

1 Introduction

As cryptography is adopted in a growing number of applications, it is often the case that implementation correctness of cryptographic libraries becomes a necessary condition for achieving overall system and application security. Implementations of cryptographic algorithms often need to satisfy the conflicting requirements of performing complex mathematical operations while achieving a high level of performance, resulting in generally hard-to-debug code. High-profile vulnerabilities have resulted from incorrect implementations of on-paper secure cryptographic schemes, such as the recent "psychic signatures" vulnerability in Java's ECDSA library (CVE-2022-21449 [CVEa]), or the buffer overflow bug in the Keccak reference implementation (CVE-2022-37454 [CVEb]).

While a long-term solution to the problem of developing robust implementations of cryptographic algorithms "from the get-go" will likely require more extensive adoption of formal verification tools, today it is still the case that initial reference implementations for cryptographic schemes are often written directly in system programming languages such as

C, rather than starting from a formal specification. Often these reference implementations are then used to generate known-answer test vectors used to verify correctness of later implementations and may also be directly deployed as part of products, potentially resulting in the proliferation of bugs (e.g., CA-1999-15 [CMU99]).

In the general area of software engineering fuzzing has proven to be a valuable strategy complementing classical testing strategies by discovering issues that human developers don't anticipate. However, most general software has highly structured inputs and outputs. This is not the case in cryptographic software, where (a subset of) the inputs and outputs should often look random, and violations of the related security properties are often not directly observable through system behaviour.

In this work, we investigate the strategy of searching for counterexamples of expected cryptographic properties of the implementations under test to automatically discover bugs in implementations of cryptographic primitives. This process can be seen as a form of metamorphic testing [ZHT+04], instantiated using fuzzing tools.

In a nutshell, our approach is to run "bit-contribution" and "bit-exclusion" tests on selected inputs to a cryptographic function. We make use of the AFL++ fuzzer [FMEH20], running on test programs specifically designed to crash whenever an expected cryptographic property of a function is not satisfied. For example, when testing IND-CCA-secure key encapsulation mechanisms' decapsulation, we may generate a valid encapsulation c, modify it into $c' \neq c$, and attempt to decapsulate it; if decapsulation succeeds, we crash the program. This approach, used on every (function, input) combination provided by the syntax of hash functions, key encapsulation mechanisms (KEMs), and digital signature schemes (DSSs), together with a cryptographically-informed test of their expected output properties and with appropriate de-randomisation, led to our discoveries.

Related work. Our starting point is the work of Mouha *et al.* [MRKK18] on hash functions proposed during the SHA3 competition run by the United States National Institute of Standards and Technologies (US NIST). We borrow their techniques and apply them to key encapsulation mechanisms and digital signature schemes, as well as running some of their tests on different hash functions. While the PQClean suite performs similar tests [KSSW22] on some KEM and DSS functions, they do so on a smaller selection of algorithms and only testing: the effect of replacing valid inputs with random tapes (for KEMs), whether replacing the public key with a different valid public key still allows signature verification to pass (for DSSs), and whether canary bytes around allocated buffers are touched during normal operation (for KEMs and DSSs). This implies that some of the bugs we encounter would likely not be detected.

Our work is also related to CLFuzz [ZMC+23], which also focuses on testing cryptographic libraries via fuzzers that are aware of some of the semantics of cryptographic algorithms. Therein, the fuzzing techniques are aware of the correctness semantics of cryptographic primitives, while our work tests for a wider set of semantics that our primitives should uphold. Yang, Arya and Wang [YAW23] combine formal methods with fuzzing in order to discover bugs in complex systems such as 5G implementations. Our approach limits its scope to the underlying cryptographic primitives, rather than aiming to holistically target an entire system.

Our contributions. During the development of the project, we identified various cryptographic bugs, including: an IND-CCA bug for some parameter sets in the reference implementation of NTRU (fixed in 2020 after disclosure); a bug breaking output consistency of the reference implementation of the KNOT-384 hash function submitted to the first round of the NIST Lightweight Cryptography Standardization process (independently fixed by the authors, albeit not explicitly disclosed), that would allow influencing output digests by modifying bytes neighbouring (but not belonging to) the input buffer; second-preimage

bugs in the AVX/SSE implementations of acehash and syconhash submitted to the NIST Lightweight Cryptography Standardization process. We remark that the bugs mentioned exclusively affect the specific implementations tested for each scheme, rather than their mathematical design.

Our tests also automatically detected a few surprising properties, including: the lack of strong unforgeability for the compressed signature format in the Falcon DSS (in version 1.1 of its spec, independently discovered by the Falcon team); that Rainbow and Falcon public keys pk could be modified such that verification using a specific $pk' \neq pk$ would still succeed (as of LibOQS 0.4.0, since resolved), a property of many KEMs using implicit-rejection Fujisaki-Okamoto transform variants where valid ciphertexts decapsulate correctly under modified secret keys; a similar property for the randomized version of the Dilithium DSS, where valid signatures can be generated by modified secret keys in the presence of a faulty randomness source. In contrast with the bugs that we previously mentioned, these properties result from the schemes' definition, and were identified by automatically testing an implementation. While these properties do not invalidate the security notions claimed (IND-CCA and EUF-CMA), some may be unexpected by users and could affect the interaction between the cryptographic primitives and the protocols these may be used as components of, similarly to what Jackson *et al.* [JCCS19] discover for non-post-quantum signature schemes using formal verification tools.

Finally, we develop a syntax for the testing procedure, which, once implemented, lets us reuse a majority of the testing code across different primitives, libraries, and fuzzers, at minimal implementation cost. This syntax could easily be adapted to check for other properties that we did not test, such as near-collisions in hash functions, which could also result from potential bugs.

We believe our results further demonstrate the usefulness of the techniques introduced by Mouha *et al.*, which are successful in identifying subtle bugs and properties of implementations and schemes in a black-box manner. We think our formalisation of it simplifies the extension to other primitives and tests, and we released our implementation on `https://github.com/jangilcher/cryptoTesting`. We think that the adoption of similar testing techniques (possibly as an optional "mode") by cryptographic algorithm testing suites such as SUPERCOP or Project Wycheproof [BDK+16] could help increase early detection of various bugs at a relatively low engineering cost for the cryptographic community.

Paper roadmap. In Section 2 we lay down preliminary notation and definitions. In Section 3 we recall how metamorphic testing can be used to stress hash function implementations, and explain how to extend this to KEMs and DSSs. In Section 4 we introduce our approach to implementing metamorphic testing on cryptographic schemes, and describe the specific tests we run. In Section 5 we describe the results of our experiments, the bugs we identify as well as some possibly unexpected properties found.

2 Preliminaries

We denote by $\mathbb{Z}_{\geq 0}$ the set of non-negative integers. We define $[n] \coloneqq \{1, 2, \ldots, n\}$. Let 'true' and 'false' be denoted by $\top$ and $\bot$, respectively. Given a statement S, we denote its truth value by $[\![S]\!]$. Given a bit string s, we denote its bitlength as $|s|$. Given two strings s_1 and s_2, we denote $s_1||s_2$ as the concatenation of s_1 and s_2. We write $\oplus$ for the exclusive-or operation on single bits or on bitstrings having the same length. We write $(0^x 10^y)_2$ to represent the bitstring of length $x + y + 1$ with all bits set to zero except for the bit in position $x + 1$, which is set to one. Whenever we write $m \bmod n$ we mean the value m' in $\{0, \ldots, n-1\}$ such that $m = m' + q \cdot n$ for some $q \in \mathbb{Z}$. We denote integer

floor division of x by y as $\lfloor x/y \rfloor$. We write $(t_i)_{i \in S}$ to denote a sequence of elements t_i with $i \in S \subset \mathbb{Z}$, and omit S when clear for context, and similarly for sets $\{t_i\}_{i \in S} \coloneqq \{t_i \mid i \in S\}$.

In pseudocode, we denote variable assignment using $\leftarrow$, and random sampling from a set using $\leftarrow\!\!\$$. Whenever we are addressing a buffer x starting from the second byte, rather than the first, we write "$\&x[1]$" rather than "x".

In the following definitions we omit the random tape input for randomized algorithms. In contexts where the random tape is important we will explicitly add it as last argument separated using a semicolon; e.g. encapsulation using a random tape r outputs $\mathsf{Encaps}(\mathsf{pk}, m; r)$. We follow the standard notation for security parameters, using unary representation so that the runtime of the key-generation Turing machine is polynomial in the length of its input tape. If a quantity $Q(\lambda)$ is negligible in λ, we may write $Q \in \mathsf{negl}(\lambda)$.

Definition 1 (Stateful pseudorandom generator)**.** We define a "stateful" pseudorandom generator PRG to be a function mapping λ-bit "seeds" $s \in \{0,1\}^\lambda$ into random tapes $r \in \{0,1\}^*$ and new seeds, $\mathsf{PRG}\colon \{0,1\}^\lambda \to \{0,1\}^\lambda \times \{0,1\}^*$.

We chose this syntax for PRGs to simplify describing our generation of random tapes during tests in Section 4.

Definition 2 (Key Encapsulation Mechanism (KEM))**.** Let Π be a triple $(\mathsf{Gen}, \mathsf{Encaps}, \mathsf{Decaps})$ of algorithms where $\mathsf{Gen}(1^\lambda) \to (\mathsf{pk}, \mathsf{sk})$ takes a security parameter and outputs a public-secret key pair, $\mathsf{Encaps}(\mathsf{pk}) \to (c, ss)$ takes a public key pk and outputs a ciphertext c and a shared secret ss, and $\mathsf{Decaps}(\mathsf{sk}, c) \to ss$ is a deterministic algorithm that takes a secret key sk and a ciphertext c and outputs a shared secret ss (or a distinguished failure symbol $\perp$). We say Π is a *key encapsulation mechanism* (KEM), if

$$1 - \Pr\left[(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{Gen}(1^\lambda), (c, ss) \leftarrow \mathsf{Encaps}(\mathsf{pk}), ss' \leftarrow \mathsf{Decaps}(\mathsf{sk}, c)\colon ss = ss'\right] \in \mathsf{negl}(\lambda).$$

Definition 3 (Digital Signature Scheme (DSS))**.** Let $\Pi = (\mathsf{Gen}, \mathsf{Sign}, \mathsf{Verify})$ be a triple of algorithms where $\mathsf{Gen}(1^\lambda) \to (\mathsf{pk}, \mathsf{sk})$ takes a security parameter and outputs a public-secret key pair, $\mathsf{Sign}(\mathsf{sk}, m) \to \sigma$ takes a secret key sk, a message $m \in \{0,1\}^*$ and outputs a signature σ, and $\mathsf{Verify}(\mathsf{pk}, m, \sigma) \in \{0,1\}$ is a deterministic algorithm that takes a public key pk, a message $m \in \{0,1\}^*$ and a signature σ and outputs a bit b, with $b = 1$ meaning "valid" and $b = 0$ meaning "invalid". We say Π is a *digital signature scheme* (DSS) if

$$1 - \Pr\left[(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{Gen}(1^\lambda), \sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)\colon \mathsf{Verify}(\mathsf{pk}, m, \sigma) = 1\right] \in \mathsf{negl}(\lambda).$$

Return values. We note that the syntax for public-key encryption [Gol04, Def. 5.1.1], key encapsulation mechanism [KL20, Def. 12.9], and digital signatures [Gol04, Def. 6.1.1] used in provable cryptography does not consider "return values". However, implementations of these primitives often do make semantic use of return values to signal successful operation. We consider relevant acknowledging return values, and therefore will add them to the output of cryptographic functions, even when their cryptographic syntax does not include it (e.g., meaning that implementations of Gen return $(\mathsf{pk}, \mathsf{sk}, rv)$ rather than just $(\mathsf{pk}, \mathsf{sk})$).[1]

2.1 Security notions

In this section we recall security notions for hash functions, KEMs and DSSs, that will be "stressed" by our testing, meaning that the tests may output counterexamples to the notion, demonstrating that a specific implementation does not satisfy it. This is not meant as an exhaustive list of security properties.

[1] It should be pointed out that LibOQS' C API documentation mentions that the return values of cryptographic algorithms capture whether the algorithm did return or whether an unexpected error occurred, https://openquantumsafe.org/liboqs/api/common.html#common_8h_1a7682cde206b02be2addea22bb31cf7fc. Yet, this does not appear to be the case in practice: on the one hand, signature verification uses return values to communicate whether a signature was valid or not, on the other, various KEMs use the return value to communicate "explicit" rejection.

Definition 4 (Second-preimage resistance)**.** Let $H\colon \{0,1\}^m \to \{0,1\}^\ell$ be a hash function. H is said to be *second-preimage resistant* if for any efficient adversary $\mathcal{A}$ and random message $x \leftarrow\!\!\$\ \{0,1\}^m$,

$$\Pr[x \leftarrow\!\!\$\ \{0,1\}^m, x' \leftarrow \mathcal{A}(x)\colon H(x) = H(x') \text{ and } x \neq x'] \in \mathsf{negl}(\lambda).$$

Definition 5 (KEM IND-CCA security)**.** Let $\Pi = (\mathsf{Gen}, \mathsf{Encaps}, \mathsf{Decaps})$ be a KEM. We say that Π has *indistinguishable encapsulations under chosen ciphertext attacks* (IND-CCA) if for any efficient adversary $\mathcal{A}$

$$\Pr\left[b \leftarrow\!\!\$\ \{0,1\}, (\mathsf{pk},\mathsf{sk}) \leftarrow \mathsf{Gen}(1^\lambda), (c^*, ss^*) \leftarrow \mathsf{Encaps}(\mathsf{pk}), b' \leftarrow \mathcal{A}^{O^{\mathsf{Decaps}}_{\mathsf{sk}}}(\mathsf{pk}, c^*, \overline{ss})\colon\ b = b'\right]$$

is negligible, where $O^{\mathsf{Decaps}}_{\mathsf{sk}}(c)$ returns $\mathsf{Decaps}(\mathsf{sk}, c)$ except if $c = c^*$, in which case it returns $\perp$, and $\overline{ss} = ss^*$ if $b = 0$ and is randomly sampled from the shared secret space if $b = 1$.

Definition 6 (sUF-CMA security)**.** Let $\Pi = (\mathsf{Gen}, \mathsf{Encaps}, \mathsf{Decaps})$ be a DSS. We say that Π is *strongly unforgeable under chosen message attacks* (sUF-CMA) if for any efficient adversary $\mathcal{A}$

$$\Pr\left[(\mathsf{pk},\mathsf{sk}) \leftarrow \mathsf{Gen}(1^\lambda), (m^*, \sigma^*) \leftarrow \mathcal{A}^{O^{\mathsf{Sign}}_{\mathsf{sk}}}(\mathsf{pk})\colon \mathsf{Verify}(\mathsf{pk}, m^*, \sigma^*) = 1\right]$$

is negligible, where $O^{\mathsf{Sign}}_{\mathsf{sk}}(m)$ returns $(m, \mathsf{Sign}(\mathsf{sk}, m))$ and did never output (m^*, σ^*).

3 Metamorphic testing

By design, the output of cryptographic primitives is often conjectured to be indistinguishable from random. When writing a new implementation P of a cryptographic primitive π, a common testing methodology is differential testing: compare the output of P with that of a reference implementation P_0 deemed correct. This can be done by generating test cases $(t_i)_i$, and checking that $P(t_i) = P_0(t_i)$.

This approach however poses the problem of testing the initial reference implementation P_0. Indeed, given an input t_i, it may be impossible to tell if $P_0(t_i)$ is correct, due to its pseudorandomness. If π is an encryption scheme, for example, one may test whether ciphertexts generated with P_0 correctly decrypt. However, this may not be a sufficient criterion if multiple ciphertexts could plausibly decrypt to the same message. If π is a one-way function, such as a hash function, this may be even harder, since no inverse function would be known. This is an instance of the "oracle problem" in software testing [Wey82].

We say that a test input t_i is *successful* if, given a known output $\pi(t_i)$, an implementation under test satisfies $P(t_i) = \pi(t_i)$. We say t_i is *indeterminate* if $P(t_i)$ does not obviously fail (say, by crashing the program). We say t_i is *unsuccessful* if $P(t_i) \neq \pi(t_i)$. Metamorphic testing [CCY20, CTZ03] is a testing approach where a series of test cases $(t_i)_i$ that are indeterminate can be transformed into new test cases $(t'_j)_j$ that may be possible to classify as unsuccessful, given the partial knowledge of π.

For example, Mouha *et al.* [MRKK18] use metamorphic testing to test hash functions, by creating $n+1$ indeterminate test cases $(t_i)_i$ by setting $t_0 = (0^n)_2$ be the zero-bitstring of length n, and $t_i = (0^{i-1}10^{n-i})_2$ for $i \in [n]$ be the bitstring with zeroes everywhere except at index i, for $i > 0$, and checking whether $P_0(t_i) = P_0(t_j)$ for some $i \neq j$ (among other tests). Whenever this happens, a likely bug in the implementation is found, and (t_i, t_j) is marked as an unsuccessful test case, since collisions or second pre-images to the hash function π should be hard to find. This simple testing technique (the "Bit-Contribution" test, in [MRKK18]'s terminology) allowed them to identify 19 bugs in submissions to the SHA3 standardization competition.

3.1 Metamorphic testing for Hash functions

We now describe the original set of metamorphic tests introduced in [MRKK18] for cryptographic hash functions. We will use these as the initial point for our further tests on KEMs and DSSs.

Note that in order to support progressive hashing, the hash functions analysed by Mouha *et al.* [MRKK18] implement an IUF-interface.[2] Mouha *et al.* introduce three kinds of metamorphic test: the "Bit-Contribution", "Bit-Exclusion", and "Update" tests.

Bit-Contribution. The *bit-contribution* test defines a series of input bit lengths $(\ell_i)_i \subset \mathbb{Z}_{\geq 0}$, and for each ℓ_i defines an initial test case $t_{i,0} \in \{0,1\}^{\ell_i}$. After mauling this test case by defining $t_{i,j} \coloneqq t_{i,0} \oplus (0^{j-1}10^{\ell_i - j})_2$ for $0 < j \leq \ell_i$, it collects $\mathsf{Hash}(t_{i,j})$ for all i, j as above, and checks for collisions. The rationale of this test is that a collision would potentially be caused by some bit of the input not contributing to the output in the implementation, likely meaning a bug is present in the implementation.[3] Of the 86 reference implementations tested in [MRKK18], 19 fail this test.

Bit-Exclusion. The *bit-exclusion* test targets the fact that NIST required the SHA3 competition submitters to define their hash functions over non-multiple-of-8 bit lengths. However, in practice, implementations in most languages work at byte-level, including the reference implementations that were requested to be written in the *C* programming language. Therefore, if a function was called on an input of bitlength $\ell = 6 \bmod 8$, the last 2 bits of the input *byte buffer* should be ignored. The bit-exclusion test is designed to check this property, by defining input test cases $t_{i,0}$ for $\ell_i \in \{\ell \in \mathbb{Z}_{\geq 0} \mid \ell \bmod 8 \neq 0\}$, and then mauling these into further test cases $t_{i,j} \coloneqq (t_{i,0} || 0^{8-(\ell_i \bmod 8)}) \oplus (0^{\ell_i} 0^{j-1} 10^{8-(\ell_i \bmod 8)-j})_2$, for $0 < j \leq 8 - (\ell_i \bmod 8)$. For each i, it then checks for non-collisions in $\{\mathsf{Hash}(t_{i,j})\}_j$, essentially checking if the last $8 - (\ell_i \bmod 8)$ bits are contributing to the hash when they shouldn't. If given some length ℓ_i, non-collisions are generated, this means that the implementation of Hash is incorrectly over-reading bits beyond the input boundary when producing the final digest. Of the 86 reference implementations tested in [MRKK18], 17 fail this test.

Update. The *update* test makes use of the guarantee on the internal API of Hash, by checking whether $\mathsf{Hash}(x) = \mathsf{Final}(\cdot) \circ \mathsf{Update}(\cdot, \ell_2) \circ \mathsf{Update}(\cdot, \ell_1) \circ \mathsf{Init}(x)$ for various values of x, such that $\ell_1 + \ell_2 = |x|$. Of the 86 implementations tested in [MRKK18], 32 fail.

3.2 Metamorphic testing for KEMs and DSSs

As seen above, the core metamorphic test performed in [MRKK18] on hash functions is whether outputs collide or not, depending on whether they are expected to (bit-exclusion and update tests) or not (bit-contribution tests). The reason for only checking for collisions is that hash functions only provide a method Hash, which is expected to be one-way. Thus, one cannot check for interactions of the output of Hash with other cryptographic methods.

Extending metamorphic testing to KEMs and DSSs presents us with more functions to use compared to hashes, but also with a few syntactic differences that should be addressed.

Testable functions. In the case of KEMs and DSSs, both primitives provide three methods, $(\mathsf{Gen}, \mathsf{Encaps}, \mathsf{Decaps})$ and $(\mathsf{Gen}, \mathsf{Sign}, \mathsf{Verify})$ respectively, presenting structure when composed with each other. We can thus use bit-contribution and bit-exclusion ideas

[2] "Init, Update, Final".

[3] While a fundamental weakness of the design could be possible, this does not seem to be the case for any of the tested primitives.

to stress various aspects and security properties of these primitives. Further, more tests will have to be run when testing each function. Unfortunately, for the purpose of PQC standardization, NIST did not mandate a specific internal API. Therefore, we don't define any "update" tests, unlike [MRKK18].

Randomness. Unlike hash functions, Gen, Encaps and Sign functions use random tapes as one of their inputs. This adds some variability to testing, which can be resolved by fixing a PRG seed for the random tape. However, the use of random tapes also suggests a different test: providing bad randomness. Since LibOQS does allow defining a custom PRG, this is straight forward to implement. While the setting of having access to a bad randomness source is usually out-of-model in terms of cryptographic vulnerabilities, by performing this test we do identify some implementations that potentially hang if they get an unlucky random tape, which likely is not the desired behaviour.

Fixed-length inputs. Unlike hash functions, inputs to KEM and DSS methods are often fixed-length. Exceptions are (in principle) random tapes, where however we provide fixed-lengths PRG seeds, messages input to Sign, where we consider fixed messages since these are usually first hashed,[4] and Verify for schemes where variable-length signatures are possible. In the latter case, we store the signature in a buffer large enough to always contain the largest possible signature σ output by Sign, together with a buffer encoding the byte length ℓ_σ of the specific signature. We then consider the case of mauling both σ and ℓ_σ.

Format strings. Finally, the testing functionality will be given an input string x, that could contain multiple function inputs concatenated (say, $x = pk||sk||m||sig$ for a signature scheme test) and a "format string" tape, that captures whether one should expect the output of the function being tested on a mauled version of x to equal or differ from the output of the tested on the original x. We remark that this is not imposed by the primitives, and is rather an implementation choice, helping reduce code duplication when writing the tests by combining bit-contribution and bit-exclusion tests into a single test.

4 Implementing metamorphic testing

An objective of our work is to reduce the friction for maintainers of cryptographic libraries in order to incorporate metamorphic testing techniques in said libraries. In order to achieve this goal, we developed a metamorphic framework that is general enough to encompass our test suite. In this section, we formalize this framework, and in Sections 4.1.1 to 4.1.3 we describe how it will be instantiated to test the corresponding primitives.

Our testing framework follows similar steps to those used in software fuzzing. First, we generate some valid input x with respect to some cryptographic primitive function Π and some public parameters pp. We then call Π on this input to obtain some baseline result y. We will apply some mutations to x in order to obtain a new input x'. On this mauled input we will apply again the tested function and receive a new output y'. Then, we will compare this y and y' to verify that the input mutation has caused the output to change (or not) in the way that we expected. In general, we assume non-malleability, meaning that we expect $\Pi(x) \neq \Pi(x')$. In formalizing this, we define the metamorphic test specification as follows.

Definition 7 (metamorphic test specification)**.** Let $\Phi = (\mathsf{GenInput}, \mathsf{Call}, \mathsf{Maul}, \mathsf{Match})$ be a tuple of functions with the following syntax.

[4] Testing messages of different lengths would realistically only stress the underlying hash function being used by the DSS.

- GenInput : $(\Pi, \text{pp}, \text{fmt}) \mapsto (x, y, \text{aux})$, takes an argument Π that specifies a cryptographic primitive's method implementation and how to execute it, some fixed public parameter pp, and a formatting specifier fmt and returns a triple consisting of an input x, an output y, and some auxiliary information aux.

- Call : $(\Pi, x, \text{aux}, \text{pp}, \text{fmt}) \mapsto y$, takes Π, pp, fmt as in GenInput, auxiliary information aux and an input x, and returns an output y.

- Maul : $(x, \text{fmt}, \sigma) \mapsto (x', \sigma', \text{expected\_result})$, takes an input x, a format string fmt, some state σ and outputs a mutated input x', some updated state σ' and a value expected_result that specifies the expected change that the mutated input should have on the result of Call.

- Match : $(y, y', \text{expected\_result}) \mapsto b$, compares y with y' on account of expected_result and raises an error on failure.

Given a metamorphic test specification Φ, we instantiate our testing framework as shown in Figure 1 to obtain a testing function Test$(\Pi, \text{pp}, \text{fmt}, \text{runs})$.

```
Test(Π, pp, fmt, runs, σ_0)
 1  sus ← [ ]; σ ← σ_0;
 2  (x, y, aux) ← GenInput(Π, pp, fmt)
 3  // Fuzzer loop
 4  foreach i in {1, ..., runs}
 5    x', σ', expected_result ← Maul(x, fmt, σ)
 6    σ ← σ'
 7    try
 8      y' ← Call(Π, x', aux, pp, fmt)
 9      Match(y, y', expected_result)
10    catch
11      sus.append((x, fmt, σ))
12  return sus
```

```
GenInput(Π, pp, fmt)
1  x ← // an initial valid input to Π
2  aux ← // data not being mauled
3  y ← Call(Π, x, aux, pp, fmt)
4  return (x, y, aux)
```

```
Call(Π, x, aux, pp, fmt)
1  return // output of Π on x
```

```
Maul(x, fmt, σ)
1  return // a format-aware mutated x'
```

```
Match(y, y', expected_result)
1  // checks whether y' is as expected
2  // and crashes otherwise
```

Figure 1: Generic test framework.

The advantage of defining a test specification, and strictly adhering to the defined interface during its implementation is that the code implementing functions that do not have a cryptographic primitive's method implementation Π as input (e.g. Match) can be reused "as is" for different functions Π being tested. Furthermore, we observe that in practice often some of the structure and code in Π-dependent functions such as Call and GenInput can also be reused between different tests.

Given a test specification Φ, the Test function that we use, only checks for single (y, y') pairs, rather than collecting sets of multiple $\{y_0, y_1, y_2, \dots\}$ for collisions, as done in [MRKK18]. While manually implementing such a test should be possible, the reason we define the Test over pairs is that it lends itself to a trivial adaptation to fuzzing libraries. In particular, we will use Φ to define a custom AFL++ mutator function that will allow us to use the parallelization utilities provided by AFL++ while running our tests. Using

our Maul function, we essentially limit AFL++ to a subset of its mutators.[5] Whenever the Match function observes an unexpected result, we purposely crash it, causing AFL++ to register the unexpected output.

While defining a testing specification helps us normalize terminology and in principle implement more general testing, our main contribution is defining specific *cryptographically-informed* programs to fuzz. Notably, these programs are designed to crash upon violation of a cryptographic property by the underlying implementations, detecting otherwise non-crashing logical flaws.

Types. We implement our test code in the C programming language, since the reference implementations in LibOQS and SUPERCOP are similarly written in C. To keep our test code in line with the test specification Φ, we define some abstract types for inputs x (`in_t`), outputs y (`out_t`), public parameters pp (`pp_t`), auxiliary information aux (`aux_t`), formatting specifiers fmt (`fmt_t`) and expected results (`exp_res_t`).

- The method implementation Π and the public parameters `pp_t` will be specific to the cryptographic library being tested. In LibOQS' case, Π is implicitly imported with the library (`#include <oqs/oqs.h>`), while `pp_t` only contains an integer "algorithm identifier" entry which is used by the LibOQS interface to select the algorithm being used. As such the pseudocode definition of our tests will omit defining the parameter pp, since it is purely an artefact from the C implementation.
- `in_t` is a struct pointing to a byte buffer and storing its byte length.
- `out_t` is a struct pointing to a byte buffer, storing its length, and also storing an integer return value that the method being tested may return (e.g., `0` on correct decapsulation).
- `aux_t` is a list of byte buffers, each stored together with its byte length.
- `fmt_t` is a list of (ℓ, lbl) tuples, each containing a *bit* length ℓ together with an associated "label" lbl.
- `exp_res_t` is a boolean storing whether outputs (y, y') should match or not.

The case of (ℓ, lbl) tuples contained in `fmt_t` deserves a more detailed explanation. In particular, consider the case covered in bit-exclusion tests in [MRKK18]. Suppose we are testing a hash function implementation that should be defined for bit strings with length $\ell \neq 0 \bmod 8$. Say we have an input string x of bitlength $\ell = 14$. Then x will be stored in a `in_t` struct pointing to a byte buffer of byte length 2, where the last two bits are unused.

$$x = \boxed{x_0\,x_1\,x_2\,x_3\,x_4\,x_5\,x_6\,x_7}\ \boxed{x_8\,x_9\,x_{10}\,x_{11}\,x_{12}\,x_{13}\,\square\,\square}$$

Our Maul function performs both bit-contribution and bit-exclusion tests at once by making use of the format specifier fmt. In particular, we set fmt $\leftarrow$ [(14, `DIFF`), (2, `EQ`)], and store which bit should first be flipped as part of state σ. Say, $\sigma \leftarrow 0$, where we address bits from 0 to 15. Given the initial input `in_t` x, Maul would flip x's σ^{th} bit, and set the expected result value based on whether to expect the flipped bit to cause different hashes ($\sigma < 14$) or the same hashes ($\sigma \geq 14$). In practice, whether this condition is satisfied or not (and hence whether we detected a bug or not) will be checked by Match. Utility functions for computing labels and data lengths from format strings are given in Appendix A.

In order to detect memory over-reading, we tend to define format strings that include a one-byte buffer before and after the main buffer to be used by the test program. For example,

[5] A custom mutator is necessary since AFL++ does not support deactivating individual built-in mutators.

```
Match(y, y', expected_result)
1  (d, rv) ← y
2  (d', rv') ← y'
3  eq ← d = d' ∧ rv = rv'
4  if eq ≠ expected_result then
5     abort
```

Figure 2: Match function used for the tests throughout this paper.

```
Maul(x, fmt, σ)
1  if σ < 256 :
2     x' ← σ + 1
3     lbl ← DIFF
4     expected_result ← [[lbl = EQ]] // false
5     return x', σ + 1, expected_result
```

Figure 3: Maul for random tapes used in Tests 2, 5 and 10.

in the case of the above example, we may pass fmt $\leftarrow [(8, \texttt{EQ}), (14, \texttt{DIFF}), (2, \texttt{EQ}), (8, \texttt{EQ})]$, and have the test program specifically address the input and output buffers starting from the second byte, located at "&x[1]", rather than from the first, located at "&x".

4.1 Instantiating specifications

We now look at how to concretely instantiate a metamorphic test specification. We start by looking at hash functions, which present a simpler case. We then follow by outlining the tests we perform on KEMs and DSSs.

4.1.1 Hash Functions

The security properties often expected from cryptographic hash functions are preimage, second preimage, and collision resistance. Hash functions are often modelled as random oracles, their output usually being expected to look uniformly distributed.

Let H be a hash function. The Test function defined in Fig. 1 generates pairs $(x, y \leftarrow H(x))$ and $(x', y' \leftarrow H(x'))$, and checks for validity of these pairs using a Match function. While it is unclear how to test for preimage and collision resistance using this framework, second preimage resistance is to some extent testable via an equality check between y and y' (Fig. 2), as essentially previously done in [MRKK18]. Test 1 depicts the formal definition of this test.

Near-second preimages and uniform outputs may also be testable, by using some statistical test on y and y' inside Match. However, we leave this for future work.

4.1.2 KEMs

Due to the richer API compared to hash functions, more tests can be designed for KEMs. We try to capture bugs in the three functions Gen, Encaps and Decaps, by testing the behaviour of the output when modifying each individual input.

As Gen and Encaps take a random tape input, our tests will have to deal with it. We consider two scenarios. In the first scenario, we are mauling an input that is not the random tape, say pk in Encaps. In this case, we fix a seed for the pseudorandom generator (PRG). In the second scenario, we are mauling specifically the random tape. Since mauling the seed to the PRG would likely not have any effect other than testing on different randomness, we instead define a bad PRG, that repeatedly outputs a fixed byte.

Testing Gen. The key generation algorithm Gen takes as input a random tape and outputs a secret-public key pair (sk, pk). Here we only test the function under bad randomness, Test 2, looking for collisions in output. Some implementations hang on bad random tapes;

Test: $\mathsf{Hash}(\mathsf{Maul}(x))$

$\text{fmt} = [(8, \texttt{EQ}), (\ell, \texttt{DIFF}), (8, \texttt{EQ})]$, $\sigma_0 = 0$

$\mathsf{Maul}(x, \text{fmt}, \sigma)$

1 $\ell \leftarrow \mathsf{BufBytelen}(\text{fmt})$
2 $x' \leftarrow x \oplus \left(0^{\sigma-1}|1|0^{l-\sigma}\right)$
3 $\text{lbl} \leftarrow \mathsf{GetLabel}(\sigma, \text{fmt})$
4 $\text{expected\_result} \leftarrow [\![\text{lbl} = \texttt{EQ}]\!]$
5 **return** $x', \sigma + 1, \text{expected\_result}$

$\mathsf{GenInput}(\mathsf{Hash}, \text{pp}, \text{fmt})$

1 $x \leftarrow (1^8)_2 || (0^\ell)_2 || (1^8)_2$
2 $\text{aux} \leftarrow \bot$
3 $y \leftarrow \mathsf{Call}(\mathsf{Hash}, x, \text{aux}, \text{pp}, \text{fmt})$
4 **return** x, y, aux

$\mathsf{Call}(\mathsf{Hash}, x, \text{aux}, \text{pp}, \text{fmt})$

1 $(h, rv) \leftarrow \mathsf{Hash}(\&x[1])$
2 $y \leftarrow (h, rv)$
3 **return** y

Test 1: Testing Hash, mauling input x.

Test: $\mathsf{Gen}(; \mathsf{Maul}(r))$

$\text{fmt} = [(8, \texttt{DIFF})]$, $\sigma_0 = 0$

$\mathsf{GenInput}(\mathsf{KEM}, \text{pp}, \text{fmt})$

1 $x \leftarrow (0^8)_2$
2 $\text{aux} \leftarrow \bot$
3 $y \leftarrow \mathsf{Call}(\mathsf{KEM}, x, \text{aux}, \text{pp}, \text{fmt})$
4 **return** x, y, aux

$\mathsf{Call}(\mathsf{KEM}, x, \text{aux}, \text{pp}, \text{fmt})$

1 $(\mathsf{sk}, \mathsf{pk}, rv) \leftarrow \mathsf{KEM.Gen}(; x)$
2 $y \leftarrow (\mathsf{sk}||\mathsf{pk}, rv)$
3 **return** y

Test 2: Testing Gen, mauling random tape r.

this is likely due to key generation trying to sample random elements with some given mathematical property, and failing to find one.

Testing Encaps. The encapsulation function Encaps takes in input a public key pk and a random tape, and outputs a shared secret ss and an encapsulation c of ss. We run two experiments where pk is mauled. In the first one, Test 3, we test whether the shared secret generated by Encaps on a valid public key pk and on a mauled version pk′ differ. In the second experiment, Test 4, we test whether the shared secret generated by Encaps on a mauled public key still decapsulates correctly. In the experiment where we maul the random tape, Test 5, we check that outputs of Encaps change as a result.

Testing Decaps. The decapsulation function Decaps produces a shared secret ss from a secret key sk and encapsulation c. In this case, no random tape is present, and the mauling is simply performed over the secret key sk, Test 6, or the ciphertext c, Test 7, with Match checking whether the resulting shared secrets differ from the original. In particular, failing to pass the test mauling c highlights a failure to provide IND-CCA security (Def. 5).

4.1.3 DSSs

The API of digital signatures is similarly rich to that of KEMs, hence we follow a similar approach in writing and running tests. While we test the functionality of Verify, we note that in the older versions of LibOQS the interface did use instead an "Open" function (`crypto_sign_open(...)`) required by NIST. We have decided to omit also testing Open, and following to the UF-CMA syntax instead.

Testing Gen. This is done identically to what is done for KEMs.

```
Maul(x, fmt, σ)
1  ℓ ← BufBytelen(fmt)
2  if σ = −3
3     x′ ← x
4     lbl ← EQ
5  elseif σ = −2
6     x′ ← 0^l
7     if [[∃ 0 ≤ i < ℓ s.t. GetLabel(i, fmt) = DIFF ∧ x′[i] ≠ x[i]]]
8        lbl ← DIFF
9     else
10       lbl ← EQ
11 elseif σ = −1
12    x′ ← 1^l
13    if [[∃ 0 ≤ i < ℓ s.t. GetLabel(i, fmt) = DIFF ∧ x′[i] ≠ x[i]]]
14       lbl ← DIFF
15    else
16       lbl ← EQ
17 else
18    x′ ← x ⊕ (0^(σ−1)|1|0^(l−σ))
19    lbl ← GetLabel(σ, fmt)
20 expected_result ← [[lbl = EQ]]
21 return x′, σ + 1, expected_result
```

Figure 4: Maul used in Tests 3, 4, 6 to 9, 11 and 12.

```
Test: Encaps(Maul(pk); r)

fmt = [(8, EQ), (|pk|, DIFF), (8, EQ)], σ0 = −3

GenInput(KEM, pp, fmt)
1  (_, r) ← PRG("geninput")
2  (pk, sk, _) ← KEM.Gen(; r)
3  x ← (1^8)2||pk||(1^8)2
4  aux ← sk
5  y ← Call(KEM, x, aux, pp, fmt)
6  return x, y, aux

Call(KEM, x, aux, pp, fmt)
1  (_, r) ← PRG("call")
2  (c, ss, rv) ← KEM.Encaps(&x[1]; r)
3  y ← (ss||c, rv)
4  return y
```

Test 3: Testing Encaps, mauling public key pk.

```
Test: Decaps(sk, Encaps(Maul(pk); r))

fmt = [(8, EQ), (|pk|, DIFF), (8, EQ)], σ0 = −3

GenInput(KEM, pp, fmt)
1  (_, r) ← PRG("geninput")
2  (pk, sk, _) ← KEM.Gen(; r)
3  x ← (1^8)2||pk||(1^8)2
4  aux ← sk
5  y ← Call(KEM, x, aux, pp, fmt)
6  return x, y, aux

Call(KEM, x, aux, pp, fmt)
1  (_, r) ← PRG("call")
2  (c, ss_e, _) ← KEM.Encaps(&x[1]; r)
3  (ss_f, rv) ← KEM.Decaps(c, aux)
4  eq ← [[ss_e = ss_f]]
5  y ← (eq, rv)
6  return y
```

Test 4: Testing Encaps, mauling public key pk, then decapsulating.

Test: $\mathsf{Encaps}(\mathsf{pk}; \mathsf{Maul}(r))$

$\text{fmt} = [(8, \texttt{DIFF})]$, $\sigma_0 = 0$

$\mathsf{GenInput}(\mathsf{KEM}, \text{pp}, \text{fmt})$

1 $(\_, r) \leftarrow \mathsf{PRG}(\texttt{"geninput"})$
2 $(\mathsf{pk}, \mathsf{sk}, \_) \leftarrow \mathsf{KEM.Gen}(; r)$
3 $x \leftarrow (0^8)_2$
4 $\text{aux} \leftarrow \mathsf{pk}$
5 $y \leftarrow \mathsf{Call}(\mathsf{KEM}, x, \text{aux}, \text{pp}, \text{fmt})$
6 **return** x, y, aux

$\mathsf{Call}(\mathsf{KEM}, x, \text{aux}, \text{pp}, \text{fmt})$

1 $(c, ss, rv) \leftarrow \mathsf{KEM.Encaps}(\text{aux}; x)$
2 $y \leftarrow (ss||c, rv)$
3 **return** y

Test 5: Testing Encaps, mauling random tape r.

Test: $\mathsf{Decaps}(\mathsf{Maul}(\mathsf{sk}), c)$

$\text{fmt} = [(8, \texttt{EQ}), (|\mathsf{sk}|, \texttt{DIFF}), (8, \texttt{EQ})]$, $\sigma_0 = -3$

$\mathsf{GenInput}(\mathsf{KEM}, \text{pp}, \text{fmt})$

1 $(s, r) \leftarrow \mathsf{PRG}(\texttt{"geninput"})$
2 $(\mathsf{sk}, \mathsf{pk}, \_) \leftarrow \mathsf{KEM.Gen}(; r)$
3 $(\_, r') \leftarrow \mathsf{PRG}(s)$
4 $(c, ss_e, \_) \leftarrow \mathsf{KEM.Encaps}(\mathsf{pk}; r')$
5 $x \leftarrow (1^8)_2||\mathsf{sk}||(1^8)_2$
6 $\text{aux} \leftarrow (\mathsf{pk}, c)$
7 $y \leftarrow \mathsf{Call}(\mathsf{KEM}, x, \text{aux}, \text{pp}, \text{fmt})$
8 **return** x, y, aux

$\mathsf{Call}(\mathsf{KEM}, x, \text{aux}, \text{pp}, \text{fmt})$

1 $(ss_f, rv) \leftarrow \mathsf{KEM.Decaps}(\&x[1], \text{aux})$
2 $y \leftarrow (ss_f, rv)$
3 **return** y

Test 6: Testing Decaps, mauling secret key sk.

Test: $\mathsf{Decaps}(\mathsf{sk}, \mathsf{Maul}(c))$

$\text{fmt} = [(8, \texttt{EQ}), (|c|, \texttt{DIFF}), (8, \texttt{EQ})]$, $\sigma_0 = -3$

$\mathsf{GenInput}(\mathsf{KEM}, \text{pp}, \text{fmt})$

1 $(s, r) \leftarrow \mathsf{PRG}(\texttt{"geninput"})$
2 $(\mathsf{sk}, \mathsf{pk}, \_) \leftarrow \mathsf{KEM.Gen}(; r)$
3 $(\_, r') \leftarrow \mathsf{PRG}(s)$
4 $(c, ss_e, \_) \leftarrow \mathsf{KEM.Encaps}(\mathsf{pk}; r')$
5 $x \leftarrow (1^8)_2||c||(1^8)_2$
6 $\text{aux} \leftarrow (\mathsf{pk}, \mathsf{sk})$
7 $y \leftarrow \mathsf{Call}(\mathsf{KEM}, x, \text{aux}, \text{pp}, \text{fmt})$
8 **return** x, y, aux

$\mathsf{Call}(\mathsf{KEM}, x, \text{aux}, \text{pp}, \text{fmt})$

1 $(ss_f, rv) \leftarrow \mathsf{KEM.Decaps}(\text{aux.sk}, \&x[1])$
2 $y \leftarrow (ss_f, rv)$
3 **return** y

Test 7: Testing Decaps, mauling ciphertext c.

Test: $\mathsf{Sign}(\mathsf{Maul}(\mathsf{sk}), m; r)$

$\text{fmt} = [(8, \texttt{EQ}), (|\mathsf{sk}|, \texttt{DIFF}), (8, \texttt{EQ})]$, $\sigma_0 = -3$

$\mathsf{GenInput}(\mathsf{SIG}, \text{pp}, \text{fmt})$

1 $(\_, r) \leftarrow \mathsf{PRG}(\texttt{"geninput"})$
2 $(\mathsf{pk}, \mathsf{sk}, \_) \leftarrow \mathsf{SIG.Gen}(; r)$
3 $m \leftarrow (0^{256})_2$
4 $\text{aux} \leftarrow (\mathsf{pk}, m)$
5 $x \leftarrow (1^8)_2||\mathsf{sk}||(1^8)_2$
6 $y \leftarrow \mathsf{Call}(\mathsf{SIG}, x, \text{aux}, \text{pp}, \text{fmt})$
7 **return** x, y, aux

$\mathsf{Call}(\mathsf{SIG}, x, \text{aux}, \text{pp}, \text{fmt})$

1 $(\_, r) \leftarrow \mathsf{PRG}(\texttt{"call"})$
2 $(\ell_\sigma, \sigma, rv) \leftarrow \mathsf{SIG.Sign}(\&x[1], \text{aux}.m; r)$
3 $y \leftarrow (\ell_\sigma||\sigma, rv)$
4 **return** y

Test 8: Testing Sign, mauling secret key sk.

Test: $\mathsf{Sign}(\mathsf{sk}, \mathsf{Maul}(m); r)$

fmt $= [(8, \texttt{EQ}), (256, \texttt{DIFF}), (8, \texttt{EQ})]$, $\sigma_0 = -3$

$\mathsf{GenInput}(\mathsf{SIG}, \text{pp}, \text{fmt})$

```
1  (_, r) ← PRG("geninput")
2  (pk, sk, _) ← SIG.Gen(; r)
3  aux ← sk
4  m ← (0^256)_2
5  x ← (1^8)_2||m||(1^8)_2
6  y ← Call(SIG, x, aux, pp, fmt)
7  return x, y, aux
```

$\mathsf{Call}(\mathsf{SIG}, x, \text{aux}, \text{pp}, \text{fmt})$

```
1  (_, r) ← PRG("call")
2  (ℓ_σ, σ, rv) ← SIG.Sign(aux.sk, &x[1]; r)
3  y ← (ℓ_σ||σ, rv)
4  return y
```

Test 9: Testing Sign, mauling message m.

Test: $\mathsf{Sign}(\mathsf{sk}, m; \mathsf{Maul}(r))$

fmt $= [(8, \texttt{DIFF})]$, $\sigma_0 = 0$

$\mathsf{GenInput}(\mathsf{SIG}, \text{pp}, \text{fmt})$

```
1  (_, r) ← PRG("geninput")
2  (pk, sk, _) ← SIG.Gen(; r)
3  x ← (0^8)_2 // set random tape for
4  // SIG.Sign to a repeated 0-byte
5  m ← 00 // 1 byte
6  aux ← (sk, m)
7  y ← Call(SIG, x, aux, pp, fmt)
8  return x, y, aux
```

$\mathsf{Call}(\mathsf{SIG}, x, \text{aux}, \text{pp}, \text{fmt})$

```
1  (ℓ_σ, σ, rv) ← SIG.Sign(aux.sk, aux.m; x)
2  y ← (ℓ_σ||σ, rv)
3  return y
```

Test 10: Testing Sign, mauling random tape r.

Test: $\mathsf{Verify}(\mathsf{Maul}(\mathsf{pk}), m, \sigma)$

fmt $= [(8, \texttt{EQ}), (|\mathsf{pk}|, \texttt{DIFF}), (8, \texttt{EQ})]$, $\sigma_0 = -3$

$\mathsf{GenInput}(\mathsf{SIG}, \text{pp}, \text{fmt})$

```
1  (s, r) ← PRG("geninput")
2  m ← 42^32 // 32 bytes
3  (sk, pk, _) ← SIG.Gen(r)
4  (_, r') ← PRG(s)
5  (ℓ_σ, σ, _) ← SIG.Sign(sk, m; r')
6  x ← (1^8)_2||pk||(1^8)_2
7  aux ← (sk, m, σ, ℓ_σ)
8  y ← Call(SIG, x, aux, pp, fmt)
9  return x, y, aux
```

$\mathsf{Call}(\mathsf{SIG}, x, \text{aux}, \text{pp}, \text{fmt})$

```
1  rv ← SIG.Verify(&x[1], aux.m, aux.σ)
2  y ← (rv, rv)
3  return y
```

Test 11: Testing Verify, mauling public key pk.

Test: $\mathsf{Verify}(\mathsf{pk}, \mathsf{Maul}(m), \sigma)$

fmt $= [(8, \texttt{EQ}), (256, \texttt{DIFF}), (8, \texttt{EQ})]$, $\sigma_0 = -3$

$\mathsf{GenInput}(\mathsf{SIG}, \text{pp}, \text{fmt})$

```
1  (s, r) ← PRG("geninput")
2  m ← (0^256)_2
3  (sk, pk, _) ← SIG.Gen(r)
4  (_, r') ← PRG(s)
5  (ℓ_σ, σ, _) ← SIG.Sign(sk, m; r')
6  x ← (1^8)_2||m||(1^8)_2
7  aux ← (pk, sk, σ, ℓ_σ)
8  y ← Call(SIG, x, aux, pp, fmt)
9  return x, y, aux
```

$\mathsf{Call}(\mathsf{SIG}, x, \text{aux}, \text{pp}, \text{fmt})$

```
1  rv ← SIG.Verify(aux.pk, &x[1], aux.σ)
2  y ← (rv, rv)
3  return y
```

Test 12: Testing Verify, mauling message m.

Test: $\mathsf{Verify}(\mathsf{pk}, m, \mathsf{Maul}(\sigma))$

$\text{fmt} = [(|\ell_\sigma|, \texttt{DIFF}), (8, \texttt{EQ}), (|\sigma|, \texttt{DIFF}), (8, \texttt{EQ})]$

$\sigma_0 = 0$

$\mathsf{GenInput}(\mathsf{SIG}, \text{pp}, \text{fmt})$

1 $(s, r) \leftarrow \mathsf{PRG}(\texttt{"geninput"})$
2 $(\mathsf{sk}, \mathsf{pk}, \_) \leftarrow \mathsf{SIG.Gen}(; r)$
3 $m \leftarrow 42^{32}$ // 32 bytes
4 $(\_, r') \leftarrow \mathsf{PRG}(s)$
5 $(\ell_\sigma, \sigma, \_) \leftarrow \mathsf{SIG.Sign}(\mathsf{sk}, m; r')$
6 $\text{aux} \leftarrow (\mathsf{pk}, \mathsf{sk}, m)$
7 $x \leftarrow (\ell_\sigma || (1^8)_2 || \sigma || (1^8)_2)$
8 $y \leftarrow \mathsf{Call}(\mathsf{SIG}, x, \text{aux}, \text{pp}, \text{fmt})$
9 **return** x, y, aux

$\mathsf{Call}(\mathsf{SIG}, x, \text{aux}, \text{pp}, \text{fmt})$

1 $rv \leftarrow \mathsf{SIG.Verify}(\text{aux.pk}, \text{aux}.m, \&x[\lfloor \frac{|\ell_\sigma|+7}{8} \rfloor + 1])$
2 $y \leftarrow (rv, rv)$
3 **return** y

$\mathsf{Maul}(x, \text{fmt}, \sigma)$

1 $(d, s) \leftarrow x$ // len(s) || s
2 **if** $\sigma < d$
3 $\quad d' \leftarrow \sigma$
4 $\quad x' \leftarrow (d', s)$
5 $\quad \text{lbl} \leftarrow \texttt{DIFF}$
6 **else**
7 $\quad \ell \leftarrow \mathsf{BufBytelen}(\text{fmt}) - d$
8 $\quad \sigma \leftarrow \sigma - d$
9 $\quad s' \leftarrow s \oplus \left(0^{\sigma-1} | 1 | 0^{l-\sigma}\right)$
10 $\quad x' \leftarrow (d, s')$
11 $\quad \text{lbl} \leftarrow \mathsf{GetLabel}(\sigma, \text{fmt})$
12 $\text{expected\_result} \leftarrow [\![\text{lbl} = \texttt{EQ}]\!]$
13 **return** $x', \sigma + 1, \text{expected\_result}$

Test 13: Testing Verify, mauling signature σ.

Testing Sign. The Sign function maps a secret key sk, message m, and random tape r, to a signature σ of length ℓ_σ (and a return value, in the case of LibOQS). Not all signature schemes tested have constant-size signatures. To address these cases, LibOQS provides an upper bound on the signature length, and outputs a buffer containing the signature σ and an integer containing the signature length ℓ_σ. We consider both σ and ℓ_σ as part of the signature and store and check them accordingly.

We expect the output of our tests in Tests 8 to 10 to result in differing signatures. We however notice that the Sign algorithm must not necessarily be a deterministic one. Therefore, we expect collisions for deterministic schemes whenever Sign is being tested using differing random tapes.

Testing Verify. The Verify function maps a public key pk, message m and signature (and signature length) (σ, ℓ_σ), into a return value indicating success or failure of verification. Checks are performed on the output return value rv accepting or rejecting the signature. Tests check whether mauling of the inputs results in a valid signature to the original, see Tests 11 to 13. In Test 13, where the signature is being mauled, the new signature passing verification implies a lack of strong unforgeability (Def. 6) of the implementation.

5 Our Experiments

Hardware. We run our tests across three machines. Machine A has two Intel Xeon Gold 6138 CPUs at 2.00GHz with 20 cores each and 376 GiB of RAM. Machine B has four Intel Xeon Gold 6252 CPUs at 2.10GHz with 24 cores each and 754 GiB of RAM. Machine C has one AMD EPYC 7742 CPU at 2.25GHz with 64 cores and 503 GiB of RAM.

Testing loop. We use AFL++ [FMEH20, AFL] for handling crashes of the program under observation and collecting information required to reproduce such crashes. To better

suit our needs we increased the 1 MB limit on input file size to 10 MB, which was necessary to accommodate the large public keys of some schemes. Further, we deactivate the inbuilt mutators and only use a custom single bit-flip mutator, to remove redundant mutations. Lastly, we limit the maximum number of recorded crashes down to 10 unique crashes to avoid storing terabytes of data.

We point out that the main benefits of relying on AFL++, rather than writing our own loop, are the automatic handling and storage of crashes. Likely, other fuzzers would suit this role equally well. We find the performance of AFL++ satisfactory.

Cryptographic implementations. The cryptographic implementations that we chose to test were collected from the Open Quantum Safe project (LibOQS) [SM17] and the SUPERCOP project [BL08], since both of these provide a unified interface across a variety of different schemes. For LibOQS, we tested releases 0.14.0 (July '25), 0.8.0 (June '23), and 0.4.0 (August '20), and the November '18 `nist-branch` snapshot. We chose to test several older versions of LibOQS, since the LibOQS project continuously removes schemes that dropped out over the course of the NIST PQC standardization process. Testing only the newest versions would only allow testing a small amount of schemes. Including these older versions, of course, comes with evaluating less mature implementations. We do not consider this a detriment to our evaluation, however, since we consider automatic testing methodologies to be a tool that should be already applied early on in the process, improving early-detection of errors. These four snapshots include the majority of all reference implementations submitted to NIST as part of their post-quantum standardization process [NIS16]. For hash functions, we tested the 20240107 release of SUPERCOP.

Baseline. To verify the advantages of cryptographically-informed fuzzing, we ran a set of baseline fuzzing experiments using AFL++. For this purpose, we implemented several simple non-semantically-aware harnesses around the API of LibOQS as well as the hash functions in SUPERCOP. These harnesses simply parse a binary file into the required inputs of the function under test and ensure that fixed length inputs are of the required sizes. A natural consequence of this approach is that this baseline does not include key-generation functions, as these implementations take no direct inputs.

5.1 Results

During our testing, we identify many instances of implementations deviating from our expected results. We count these instances in the following way. Say we have a scheme "XYZ" that presents two different parametrizations: "XYZ-128" and "XYZ-256", and that both parameter sets have a reference implementation in different versions of LibOQS. We count test results in different versions of LibOQS independently. We also count test results in different parametrizations independently. However, we do not count multiple test results in a specific parametrization and LibOQS version multiple times. For example, if the implementations of "XYZ-128/256" in LibOQS 0.4.0 and 0.8.0 share the same line of code that triggers a bug in both versions of LibOQS using two different inputs, we would count this 4 times: once per parameter set and per LibOQS version. We use this approach since we count total numbers automatically, instead of manually inspecting the reason for each specific unexpected result.

5.1.1 Baseline

During SUPERCOP testing, we observed segmentation faults in 4 hash function implementations. During LibOQS testing, we observed a total of 44 crashes and 2 hangs across the different versions tested, as shown in Table 1. Specifically, for KEMs we notice a

Table 1: Observed crashes and hangs for the AFL++ baseline. Hangs in parentheses.

Library	KEM		DSS		Wall time	Machine
	Encaps	Decaps	Sign	Verify		(Cores)
LibOQS 0.14.0	0	0	0	0	2h	A (31)
LibOQS 0.8.0	0	0	0	0	$0h\ 27m$	C (50)
LibOQS 0.4.0	8	0	9 (1)	0	$0h\ 57m$	C (50)
LibOQS `nist-branch` 11-2018	10	12 (1)	5	—	$0h\ 23m$	B (75)
SUPERCOP `20240107` Hash		4			$8h\ 6m$	C (50)

Table 2: Wall time required to run cryptographically-informed tests.

Library	Wall time	Machine (Cores)
LibOQS 0.14.0	$12h\ 40m$	B (75)
LibOQS 0.8.0	$14h\ 23m$	A (31)
LibOQS 0.4.0	$12h\ 43m$	A (31)
LibOQS `nist-branch` 11-2018	$8h\ 59m$	A (31)
SUPERCOP `20240107`	$6h\ 58m$	A (31)

strict decrease in the number crashes, from the oldest tested version to LibOQS 0.4.0. Indeed, all the crashes in LibOQS `nist-branch` 11-2018 were absent in LibOQS 0.4.0, with the ones observed in LibOQS 0.4.0 being found in schemes not yet present in the older `nist-branch`. For signatures, we found an increase in the number of crashes seemingly due to a regression in various variants of the Picnic scheme. Notably, we did not find any crashes or hangs for LibOQS version 0.8.0 and newer, indicating that these versions contain more mature implementations.

5.1.2 Cryptographically-Informed Fuzzing

In our testing, we identify 200+ malleabilities, where changing one function input does not result in a change in function output. We report in Table 3 the total number for each test, and in Table 2 the time required to run the tests. While not all malleabilities imply a security or software bug, some do. We also detect 48 hangs, 50 segmentation faults, 3 heap overflows, and 2 stack overflows caused by mauling one of the intended inputs. These could be seen as software vulnerabilities, depending on how exposed the API for the scheme is. We proceed to mention some examples below. We provide a full list of results and code for reproducing the tests at https://github.com/jangilcher/cryptoTesting.

Software bugs. We identified several software bugs, namely segmentation faults, stack and heap overflows, and memory over-reads.

In particular, we find a memory over-read vulnerability in the optimized implementation of the KNOT-384 [ZDY+19] hash function submitted to the first round of the NIST lightweight cryptography standardization process [Nat19]. This bug would result in messages with a multiple-of-6 byte length to be hashed to different values depending on the first byte following the message in memory. The issue was caused by incorrect masking in the conversion between least-significant-bit-first and most-significant-bit-first integer notation. This bug was fixed by the authors in a later round submission without disclosure of the bug. The version in SUPERCOP was not updated.[6]

[6]It should be noted that SUPERCOP detected "nondeterministic" output of KNOT-384 during testing, with a report being available at https://web.archive.org/web/20200810000404/https:/bench.cr.yp.to/web-impl/amd64-pmnod003-crypto_hash-knot384.html.

Table 3: Summary of the kind of malleabilities and crashes observed during our tests.

Test	Bit contribution: Mall.	Bit contribution: Crash/hang	Bit excl.: Non-mall.
SUPERCOP `20240107`			
Hash function tests:			
Test 1	3	0	3
LibOQS 0.14.0 — 07/2025			
KEM tests:			
Test 5	1	10 (hang)	0
Test 6	26	0	0
DSS tests:			
Test 8	11	0	0
Test 11	10	0	0
Test 13	25	0	0
LibOQS 0.8.0 — 06/2023			
KEM tests:			
Test 3	10	0	0
Test 4	10	0	0
Test 5	1	10 (hang)	0
Test 6	23	0	0
DSS tests:			
Test 8	3	0	0
Test 13	2	0	0
LibOQS 0.4.0 — 08/2020			
KEM tests:			
Test 3	23	8 (segf.)	0
Test 4	31	8 (segf.)	0
Test 5	1	10 (hang)	0
Test 6	51	0	0
Test 7	10	0	0
DSS tests:			
Test 8	10	3 (heap overf.) + 1 (hang)	0
Test 9	0	1 (hang)	0
Test 10	22	1 (hang)	0
Test 11	7	1 (hang)	0
Test 12	0	1 (hang)	0
Test 13	9	1 (hang)	0
LibOQS `nist-branch` 11-2018			
KEM tests:			
Test 2	3	0	0
Test 3	4	6 (segf.) + 1 (ret. $\perp$)	0
Test 4	9	7 (segf.) + 1 (hang)	0
Test 6	24	12 (segf.) + 7 (hang)	0
Test 7	3	9 (segf.) + 2 (stack overf.) + 4 (hang)	0
DSS tests:			
Test 10	6	0	0

We also include hangs in this classification though one can argue that there are instances where hangs are the expected behaviour. For instance, a hang might be preferable over simply using bad randomness. Ideally this should be avoided by routinely running health tests on the used randomness generators, but this poses its own challenges.

Cryptographic weaknesses. Three of our tests identify implementations not satisfying well-established security notions.

We observe three hash function implementations failing to achieve second-preimage resistance (Def. 4): acehash256v1 (SSE2), syconhash256v1 (AVX), syconhash256v1 (SSE).

We further identify KEM implementations not satisfying IND-CCA security (Def. 5). Some of these purposely do not provide such security, e.g. the ephemeral ThreeBears and SIKE variants in LibOQS 0.4.0, and the BIKE variant in LibOQS `nist-branch` 11-2018. Some crash or hang on decapsulation of mauled ciphertext, such as Big Quake, LedaKEM and Lima in LibOQS `nist-branch` 11-2018, providing a form of ciphertext-validity oracle. Finally, NTRU variants in LibOQS 0.4.0 (HRSS-701, HPS-2048-677, HPS-2048-509) perform decapsulation of the encapsulated shared secret, hence failing to provide IND-CCA security. This bug, caused by not checking zero-padding of non-byte-aligned encapsulations, was confirmed and fixed after private disclosure [NTR].

For signatures, we identify some schemes not providing strong unforgeability (Def. 6), such as various Picnic and Falcon instances in LibOQS 0.4.0, with Falcon still lacking strong unforgeability in LibOQS 0.14.0, as well as various parametrizations of on-ramp

signature schemes SNOVA [WCD+24] and CROSS [BBB+24] in LibOQS 0.14.0 that we disclosed. Private communication with the Falcon team confirmed that this had been independently discovered and addressed in version 1.2 of the specification. Shortly after disclosure, the version of Falcon available via LibOQS was updated [OQS]. In the case of SNOVA, the source of signature malleability we detected was in the last byte; for some parametrizations this last byte was not fully utilized, and the trailing unused bits would be accepted as valid regardless of their value. In the case of CROSS, the malleability was only present in the PQClean-compatible implementation provided by the authors [CRO], and not in the ones submitted to NIST, for which a similar test to ours was implemented. Particularly, the PQClean API requires passing a signature length in input to Verify, that was being ignored by CROSS, meaning any valid signature could have extra bytes appended and would still be accepted as valid. While strong UF-CMA is a standard security notion, the NIST competition did not require proposed schemes to satisfy it, and none of these schemes claim it. However, in the case of Falcon, the signature scheme is built using the GPV paradigm, which guarantees strong unforgeability [GPV08, Prop 6.1]. As the Falcon team did not mention this not being the case in their design document, users could expect this property to hold by default. The loss of strong unforgeability was due to the encoding scheme being used for integer coefficients in their signatures, which did not guarantee unique encodings.

Potentially unexpected properties. Most of our tests stress malleabilities allowed by standard security notions. However, some of these may be counter-intuitive at first thought. Among these, we highlight the malleability of keys.

One might expect that a secret key should be uniquely identified to lead to correct decapsulation. However, this is not true for a large number of schemes. For example, the many schemes using the Fujisaki-Okamoto transform [FO99, FO13] to compile a passively secure encryption scheme into an IND-CCA KEM. In particular, the random string appended to the secret key in the implicit-rejection variants of the transform [HHK17], is not used if a ciphertext was not altered before decapsulation. Thus, modifying this part of the secret key, still allows correct decapsulation. This phenomenon is behind all implementations claiming IND-CCA security in LibOQS 0.8.0 and 0.4.0 that our tests detect as returning an unexpected result in Test 6.

We also observe cases of malleability of DSS keys. Unlike for KEMs, the malleability of secret keys does not seem to be caused by a specific transform, but rather due to having the same secret key format among randomized and de-randomized variants of the scheme. For example, Dilithium's secret key has the format $\mathsf{sk} = (\dots, K, \dots)$ where $K \in \{0,1\}^{256}$ is used in the deterministic variant of the scheme for generating message-dependent pseudorandomness within Sign. The randomized variant of the scheme ignores K and samples such signing randomness freshly. As in our tests we fix the seed of the random number generator (providing the "fresh" signing randomness to the randomized variant), ignoring K results in different secret keys producing the same signature. Unlike for KEMs where decapsulation is inherently deterministic, this case would only be observable in deployment if the randomness source for the signing process was faulty.

In the case of public keys, we notice that being able to maul a public key pk such that, given a (pk, m, σ) tuple that passes verification, $(\mathsf{Maul}(\mathsf{pk}), m, \sigma)$ also does, is reminiscent of breaks of conservative exclusive ownership (CEO) [PS05, Def. 1]. Unlike in the case of successful CEO adversaries, we are however not able to learn a valid secret key for $\mathsf{Maul}(\mathsf{pk})$, excluding the possibility of impersonation attacks [JCCS19]. Our attacks do match the conditions of a valid attack on strong CEO (S-CEO) [CDF+21, Def. IV.1] and on strong universal exclusive ownership (S-UEO) [BCJZ21, Def. 11]. However, the extent to which this leads to practical issues in deployed protocols is unclear.

5.1.3 Discussion

In LibOQS 0.4.0, both the baseline and our approach identified segmentation faults during encapsulation of compressed variants of SIKE and SIDH. However, our approach additionally identified 10 counterexamples against IND-CCA for KEMs. For DSS, we observed 9 segmentation faults in variants of Picnic during signing in our baseline and identified 3 buffer overflows in Picnic using our proposed approach. This might suggest 6 faulty Picnic implementations were missed by our approach, however we identified all of these additional 6 variants as faulty when mauling the random tape. Similar to KEMs, we additionally identified 9 cases lacking strong unforgeability. Similarly, for `nist-branch` 11-2018 of LibOQS the baseline identified 12 segmentation faults and 1 hang during decapsulation and 5 segmentation faults during encapsulation.[7] All of these issues were also found by our approach, as well as 3 IND-CCA counterexamples. For DSS the baseline flagged Dilithium III due to encountering aborts and Picnic_L3_UR due to segmentation faults, which were missed by our approach. For versions 0.8.0 and 0.14.0 of LibOQS, we find no issues with our baseline. Cryptographically informed testing however finds 2 counterexamples to strong unforgeability in LibOQS 0.8.0 and 25 in 0.14.0. For SUPERCOP, we observed segmentation faults for acehash256v1 (AVX and SSE2 versions) and syconhash256v1 (AVX and SSE versions). In all cases these segmentation faults were caused by invalid read operations. These are likely related to the violations of second-preimage resistance we identify using our new approach. In total there are 13 IND-CCA and 36 sUF-CMA counterexamples found by our tests, which were not caught by our baseline. Our test also automatically report numerous potentially unexpected properties, such as lack of S-CEO and S-UEO, which could be unintentional. We don't expect that a different choice of baseline fuzzer would make a significant difference in terms of detecting security notion violations or unexpected properties as these do not cause crashes in themselves, and rather need a specific harness implementing our or similar tests to be detected. While our approach also caught almost every memory issue that our baseline identified, we view cryptographically-informed testing as a complementary approach to traditional fuzzing and appropriate use of compiler flags.

Conclusions. Metamorphic testing had been previously proposed as a source of stress tests for evaluating hash function implementations [MRKK18]. In this work, we show that a similar approach can be used to test implementations of more complex primitives. We demonstrate this by (re-)discovering various bugs in implementations of KEMs and DSSs adopted by the LibOQS project at different points in time. We further show that metamorphic testing enables the discovery of bugs that a traditional fuzzing approach does not discover. This indicates an avenue for future work: extending metamorphic testing to more security definitions and primitives. Further, metamorphic testing could be combined with statistical approaches such as `dudect` [RBV17] to also check for side channels, e.g. in the form of constant-time violations. Our implementation also suggests that this kind of test could relatively easily be integrated with pre-existing libraries such as SUPERCOP and LibOQS due to the common SUPERCOP API [BL08] used by these collections. This would allow implementers to quickly be made aware of bugs and other unexpected properties their implementations may present.

Acknowledgements. We thank Greg Zaverucha for many useful discussions on early versions of this work. We also thank Martin R. Albrecht for providing servers for running our experiments. Giacomo Fenzi is partially supported by the Ethereum Foundation.

[7] As well as 5 aborts. However, the segmentation faults and aborts defined identical sets of faulty schemes.

References

[AFL] AFL++. GitHub - AFLplusplus/AFLplusplus: The fuzzer afl++ is afl with community patches, qemu 5.1 upgrade, collision-free coverage, enhanced laf-intel & redqueen, AFLfast++ power schedules, MOpt mutators, unicorn_mode, and a lot more! — github.com. https://github.com/AFLplusplus/AFLplusplus. [Accessed 09-10-2025].

[BBB+24] Marco Baldi, Alessandro Barenghi, Michele Battagliola, Sebastian Bitzer, Marco Gianvecchio, Patrick Karl, Felice Manganiello, Alessio Pavoni, Gerardo Pelosi, Pao lo Santini, Jonas Schupp, Edoardo Signorini, Freeman Slaughter, Antonia Wachter-Zeh, and Violetta Weger. CROSS — Codes and Restricted Objects Signature Scheme. Technical report, National Institute of Standards and Technology, 2024. available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures.

[BCJZ21] Jacqueline Brendel, Cas Cremers, Dennis Jackson, and Mang Zhao. The provable security of ed25519: Theory and practice. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1659–1676, 2021.

[BDK+16] Daniel Bleichenbacher, Thai Duong, Emilia Kasper, Quan Nguyen, and Charles Lee. Project wycheproof. https://github.com/google/wycheproof, 2016.

[BL08] Daniel J. Bernstein and Tanja Lange. eBACS: ECRYPT benchmarking of cryptographic systems. https://bench.cr.yp.to, accessed on 29 January 2024, 2008.

[CCY20] Tsong Y Chen, Shing C Cheung, and Shiu Ming Yiu. Metamorphic testing: a new approach for generating next test cases. *arXiv preprint arXiv:2002.12543*, 2020.

[CDF+21] Cas Cremers, Samed Düzlü, Rune Fiedler, Marc Fischlin, and Christian Janson. Buffing signature schemes beyond unforgeability and the case of post-quantum signatures. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1696–1714, 2021.

[CMU99] Software Engineering Institute CMU. 1999 CERT Advisories — sei.cmu.edu. https://www.sei.cmu.edu/library/1999-cert-advisories/, 1999. [Accessed 09-10-2025].

[CRO] CROSS. Potential SUF-CMA violation · Issue #1 · CROSS-signature/CROSS-lib-oqs — github.com. https://github.com/CROSS-signature/CROSS-lib-oqs/issues/1. [Accessed 09-10-2025].

[CTZ03] T.Y. Chen, T.H. Tse, and Z.Q. Zhou. Fault-based testing without the need of oracles. *Information and Software Technology*, 45(1):1–9, 2003.

[CVEa] CVE. cve.org. https://www.cve.org/CVERecord?id=CVE-2022-21449. [Accessed 09-10-2025].

[CVEb] CVE. cve.org. https://www.cve.org/CVERecord?id=CVE-2022-37454. [Accessed 09-10-2025].

[FMEH20] Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Marc Heuse. AFL++: Combining incremental steps of fuzzing research. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)*. USENIX Association, August 2020.

[FO99] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael J. Wiener, editor, *CRYPTO'99*, volume 1666 of *LNCS*, pages 537–554. Springer, Heidelberg, August 1999.

[FO13] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. *Journal of Cryptology*, 26(1):80–101, January 2013.

[Gol04] Oded Goldreich. *Foundations of Cryptography: Volume 2, Basic Applications*. Cambridge University Press, USA, 2004.

[GPV08] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In Richard E. Ladner and Cynthia Dwork, editors, *40th ACM STOC*, pages 197–206. ACM Press, May 2008.

[HHK17] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the Fujisaki-Okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part I*, volume 10677 of *LNCS*, pages 341–371. Springer, Heidelberg, November 2017.

[JCCS19] Dennis Jackson, Cas Cremers, Katriel Cohn-Gordon, and Ralf Sasse. Seems legit: Automated analysis of subtle attacks on protocols that use signatures. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 2165–2180. ACM Press, November 2019.

[KL20] Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography*. Chapman and Hall/CRC, December 2020.

[KSSW22] Matthias J. Kannwischer, Peter Schwabe, Douglas Stebila, and Thom Wiggers. Improving software quality in cryptography standardization projects. In *2022 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pages 19–30, 2022.

[MRKK18] Nicky Mouha, Mohammad S. Raunak, D. Richard Kuhn, and Raghu Kacker. Finding bugs in cryptographic hash function implementations. *IEEE Transactions on Reliability*, 67(3):870–884, 2018.

[Nat19] National Institute of Standards and Technologies. Announcing request for nominations for lightweight cryptographic algorithms; notice. In *83 Federal Register 43656*, pages 43656—43657, August 2019. Available at https://www.federalregister.gov/d/2018-18433.

[NIS16] NIST. Submission requirements and evaluation criteria for the post-quantum cryptography standardization process, 2016. Available at https://csrc.nist.gov/CSRC/media/Projects/Post-Quantum-Cryptography/documents/call-for-proposals-final-dec-2016.pdf.

[NTR] NTRU. Ensure trailing bits of ciphertext are 0 · jschanck/ntru@6bb5239 — github.com. https://github.com/jschanck/ntru/commit/6bb52396fed494001228ca579f4c1d91ef558171. [Accessed 09-10-2025].

[OQS] OQS. Malleable Falcon signatures, may need upgrade to v1.2 spec · Issue #1315 · open-quantum-safe/liboqs — github.com. https://github.com/open-quantum-safe/liboqs/issues/1315. [Accessed 09-10-2025].

[PS05] Thomas Pornin and Julien P Stern. Digital signatures do not guarantee exclusive ownership. In *Applied Cryptography and Network Security: Third International Conference, ACNS 2005, New York, NY, USA, June 7-10, 2005. Proceedings 3*, pages 138–150. Springer, 2005.

[RBV17] Oscar Reparaz, Josep Balasch, and Ingrid Verbauwhede. Dude, is my code constant time? In *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017*, pages 1697–1702, 2017.

[SM17] Douglas Stebila and Michele Mosca. Post-quantum key exchange for the internet and the open quantum safe project. In Roberto Avanzi and Howard Heys, editors, *Selected Areas in Cryptography – SAC 2016*, pages 14–37, Cham, 2017. Springer International Publishing.

[WCD+24] Lih-Chung Wang, Chun-Yen Chou, Jintai Ding, Yen-Liang Kuan, Jan Adriaan Leegwater, Ming-Siou Li, Bo-Shu Tseng, Po-En Tseng, and Chia-Chun Wang. SNOVA. Technical report, National Institute of Standards and Technology, 2024. available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures.

[Wey82] Elaine J. Weyuker. On Testing Non-Testable Programs. *The Computer Journal*, 25(4):465–470, 11 1982.

[YAW23] Jingda Yang, Sudhanshu Arya, and Ying Wang. Formal-guided fuzz testing: Targeting security assurance from specification to implementation for 5g and beyond. *arXiv preprint arXiv:2307.11247*, 2023.

[ZDY+19] Wentao Zhang, Tianyou Ding, Bohan Yang, Zhenzhen Bao, Zejun Xiang, Fulei Ji, and Xuefeng Zhao. KNOT: Algorithm specifications and supporting document, 2019. https://csrc.nist.gov/CSRC/media/Projects/lightweight-cryptography/documents/round-2/spec-doc-rnd2/knot-spec-round.pdf.

[ZHT+04] Zhi Quan Zhou, DH Huang, TH Tse, Zongyuan Yang, Haitao Huang, and TY Chen. Metamorphic testing and its applications. In *Proceedings of the 8th International Symposium on Future Software Technology (ISFST 2004)*, pages 346–351. Software Engineers Association Xian, China, 2004.

[ZMC+23] Yuanhang Zhou, Fuchen Ma, Yuanliang Chen, Meng Ren, and Yu Jiang. Clfuzz: Vulnerability detection of cryptographic algorithm implementation via semantic-aware fuzzing. *ACM Trans. Softw. Eng. Methodol.*, 33(2), dec 2023.

A Format Parsing Utilities

```
BufBitlen(fmt)
1  bitlen ← 0
2  foreach (ℓ, lbl) ∈ fmt
3      bitlen ← bitlen + ℓ
4  return bitlen

BufBytelen(fmt)
1  len ← ⌊(BufBitlen(fmt) + 7)/8⌋
2  return len
```

```
GetLabel(i, fmt)
1  cnt ← −1
2  idx ← −1
3  while cnt < i
4      idx ← idx + 1
5      (ℓ, lbl) ← fmt[idx]
6      cnt ← cnt + ℓ
7  return lbl
```

Figure 5: Format parsing utilities.